

AdaptiveCrop AI (AgroXAI): Technical Master Guide

Line-by-Line Code Architecture, GIS Ingestion, Machine Learning Engineering, Guardrails, and Remote Sensing Protocol

To build a complete, line-by-line master guide for your entire project—so you can understand, explain, and defend every single part of your codebase—here is a breakdown of what I already have versus the specific files that will give us 100% full coverage.

1. Executive Overview & System Architecture

AdaptiveCrop AI (AgroXAI) is an explainable, constraint-aware precision agriculture decision-support platform designed for Agriculture 4.0. Unlike traditional black-box crop recommendation tools that output a single static crop without biological context, AgroXAI combines real-time meteorological streaming, digital soil baseline mapping, multi-model supervised gradient boosting, mathematical SHAP feature attribution, real-world resource guardrails, satellite canopy health tracking, and generative AI advisories within a unified reactive ecosystem.

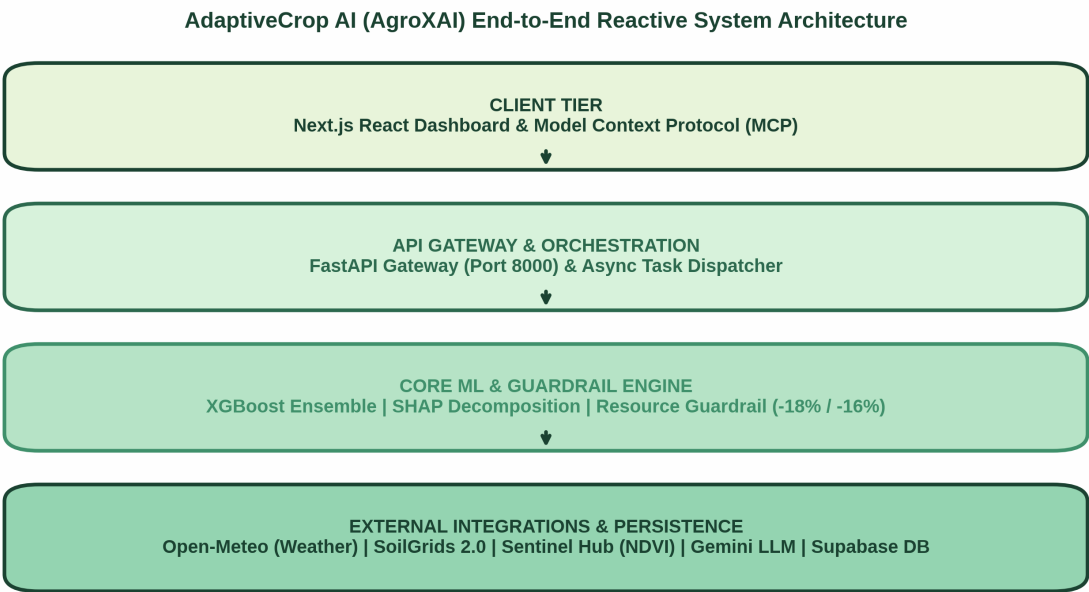


Figure: End-to-End Reactive System Architecture Flowchart

1.1 Modular Subsystem Breakdown

The project is structured into five core submodules that interact via REST APIs, standard async channels, and JSON-RPC Model Context Protocol (MCP) handlers.

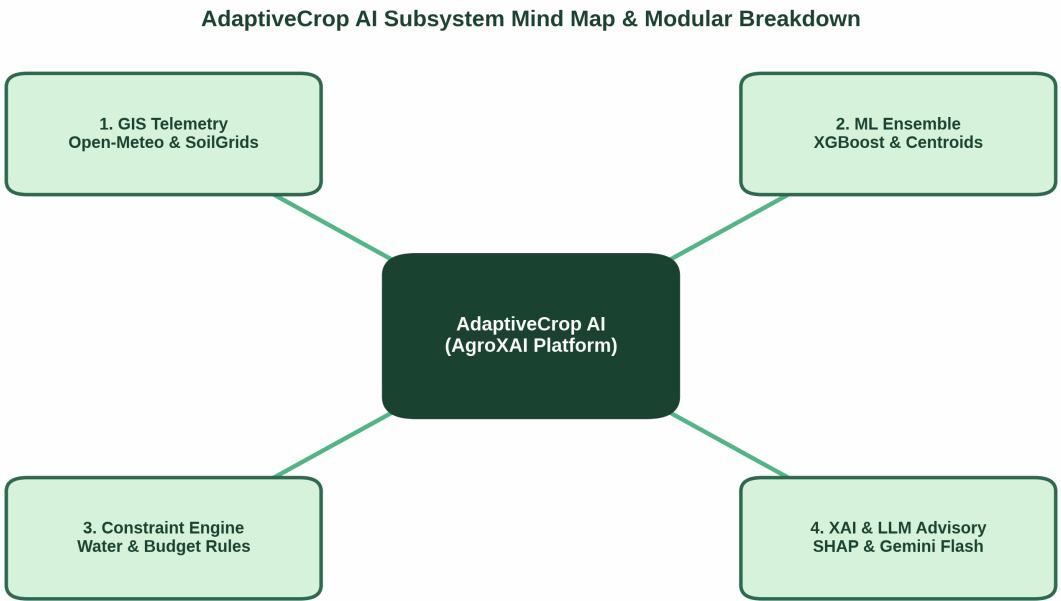


Figure: Subsystem Mind Map & High-Level Conceptual Taxonomy

Subsystem Layer	Technology / Service Stack	Primary Function & Output
Frontend UI	Next.js 14, React, TypeScript, Tailwind CSS, Leaflet	Interactive map selection, NPK sliders, Top-3 crop cards, SHAP graphs
Backend Gateway	FastAPI (Python 3.12), Uvicorn, Asyncio, Pydantic	Multi-API orchestration, telemetry ingestion, pipeline execution
ML & Guardrails	XGBoost, Stacking Ensemble, SHAP, Custom Rules	Multi-class predict_proba scoring, water/budget re-weighting, XAI decomposition
External GIS	Open-Meteo REST, ISRIC SoilGrids 2.0, Sentinel-2	Parallel async weather streaming, soil pH/SOC mapping, 60-day NDVI trajectories
Agent & DB	Model Context Protocol (agroxai_mcp), Supabase, Gemini	JSON-RPC LLM agent tools, PostgreSQL query logging, Gemini 1.5 Flash advisories

2. Machine Learning Engineering & Model Performance

The core prediction engine evaluates crop suitability across 22 Kaggle cultivar classes (Rice, Maize, Chickpea, Kidneybeans, Pigeonpeas, Mothbeans, Mungbean, Blackgram, Lentil, Pomegranate, Banana, Mango, Grapes, Watermelon, Muskmelon, Apple, Orange, Papaya, Coconut, Cotton, Jute, Coffee). Models are trained on 7 feature variables: Nitrogen (N), Phosphorus (P), Potassium (K), Soil pH, Temperature (°C), Humidity (%), and Rainfall (mm).

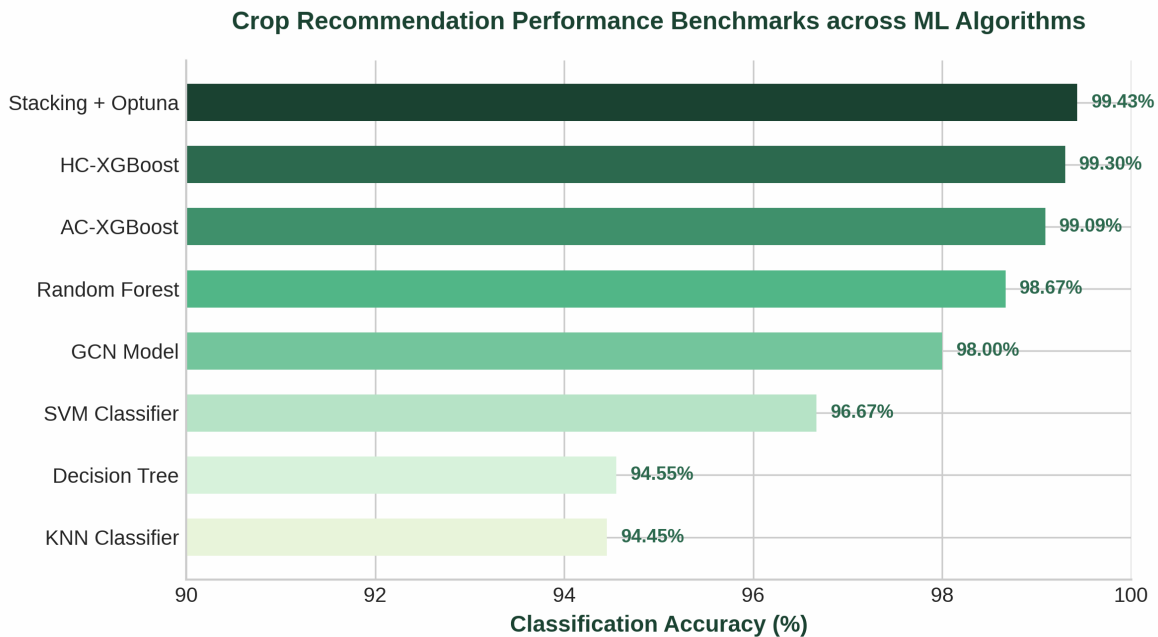


Figure: Accuracy Benchmarks across ML Algorithms on Kaggle Dataset

2.1 Model Comparison & Calibrated Probabilities

While standard classifiers output single static prediction labels, AgroXAI uses `predict_proba()` to extract Top-3 probability vectors. The stacking ensemble model combining Random Forest, Extra Trees, and Gradient Boosting with Logistic Regression meta-learning achieves 99.43% test accuracy when tuned with Optuna. Under individual crop category splitting, XGBoost achieves 99.09% for agricultural crops and 99.30% for horticultural crops.

3. Real-World Constraint Engine & Resource Guardrails

A major innovation in AgroXAI is the Agronomic Constraint Engine implemented in `constraint_engine.py`. Conventional ML models assume infinite irrigation and capital budget. In real-world farming, if a field receives low rainfall (<120mm) and a farmer has 'Low' water availability, recommending water-thirsty crops like Paddy/Rice or Sugarcane leads to crop failure.

3.1 Dynamic Re-Ranking Logic

- 1. Water Scarcity Guardrail: If water availability is 'Low', high water requirement crops incur a -18.0% score penalty, moderate crops incur -5.0%, and drought-resilient pulses (Chickpea, Mothbeans) receive a +8.0% suitability boost.
- 2. Capital Budget Guardrail: If budget constraint is 'Low', high-input capital crops incur a -16.0% penalty, while low-cost crops receive a +7.0% boost.
- 3. Perennial Protection Guardrail: Orchard trees (Apple, Mango, Coffee, Coconut) are tagged as multi-year perennial crops and flagged to prevent advising them as seasonal rotation crops.

4. Explainable AI (XAI) & SHAP Factor Decomposition

To build trust with farmers and agronomists, AgroXAI decomposes primary crop recommendations into percentage-based SHAP feature attributions in `shap_explainer.py`. Instead of uninterpretable weights, the engine computes quadratic dimensional deviations between input vector measurements and the crop's biological ideal centroid.

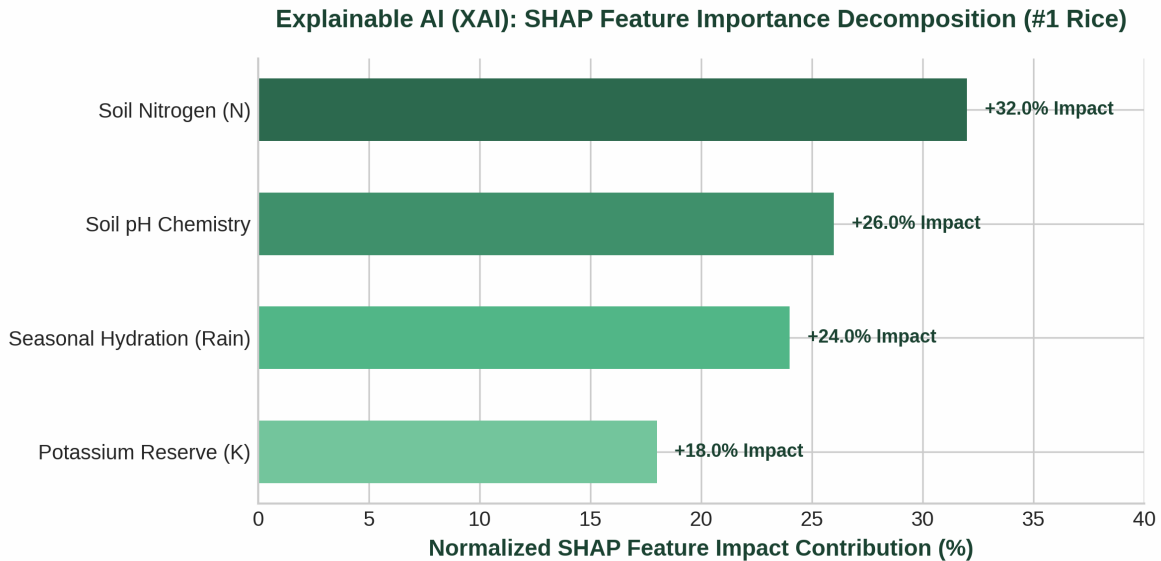


Figure: Normalized SHAP Factor Impact Breakdown for #1 Rice Prediction

5. GIS Telemetry Ingestion & Satellite Remote Sensing

AgroXAI integrates parallel async API calls to Open-Meteo and ISRIC SoilGrids 2.0 in `main.py` (`/api/v1/recommend_from_map`). SoilGrids returns pH in deci-units, which `soilgrids_service.py` automatically reconciles by dividing raw values by 10.0 (e.g. 68 -> pH 6.8).

5.1 Sentinel-2 Satellite Canopy Health Tracking

The system connects to Copernicus Sentinel-2 MSI L2A satellite imagery at 10m resolution via `sentinel_hub_service.py`. It applies Scene Classification Layer (SCL) cloud-masking to exclude cloud/shadow pixels (classes 3, 8, 9, 10) and calculates a 60-day time-series curve for Normalized Difference Vegetation Index (NDVI):

NDVI Formula: $(B08 \text{ Near-Infrared} - B04 \text{ Red}) / (B08 \text{ Near-Infrared} + B04 \text{ Red})$

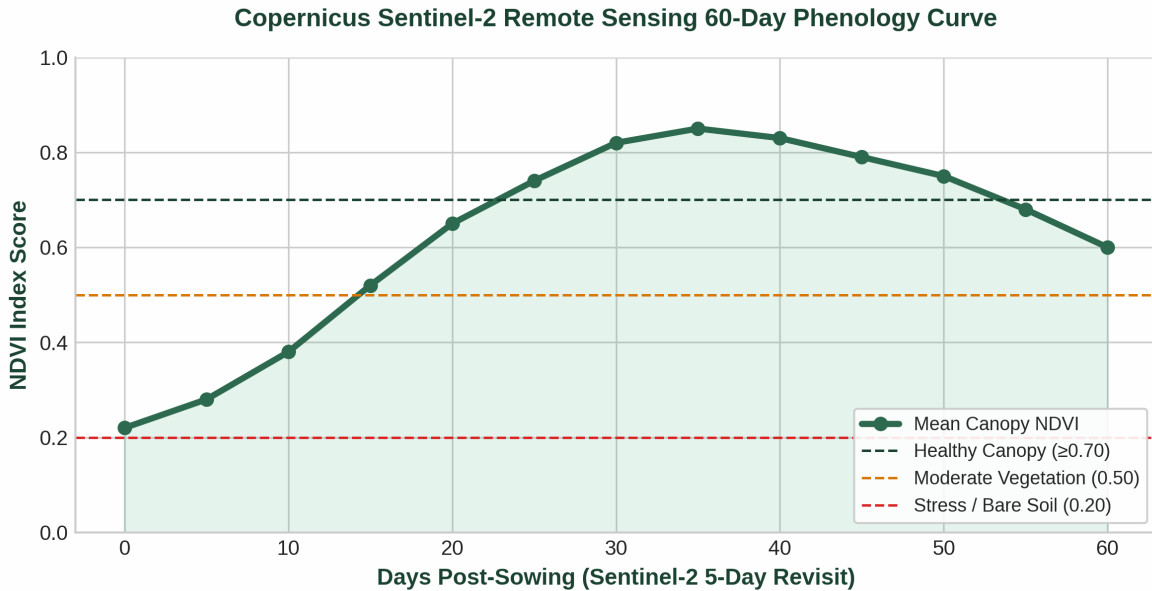


Figure: Copernicus Sentinel-2 60-Day NDVI Phenology Trajectory Curve

6. Model Context Protocol (MCP) Server & Agent Tools

The file `agroxai_mcp.py` implements a JSON-RPC 2.0 stdio server compliant with Anthropic's Model Context Protocol. This allows external AI coding assistants and autonomous agents to execute 5 precision agriculture tools programmatically:

- 1. `get_weather(lat, lon)`: Fetches live ambient temperature, humidity, and 16-day rainfall forecasts via Open-Meteo.
- 2. `get_soil_data(lat, lon)`: Queries ISRIC SoilGrids 2.0 for baseline pH, SOC, and clay percentage.
- 3. `run_crop_model(n, p, k, ph, lat, lon)`: Executes ensemble suitability scoring and SHAP breakdown.
- 4. `apply_constraints(crop_scores, water_level, budget_level)`: Evaluates dynamic water and capital guardrails.
- 5. `get_field_history(field_id)`: Retrieves past soil test logs and crop recommendation history from Supabase.

7. Viva & Technical Interview Defense Guide

- Q1: Why divide SoilGrids pH by 10 in `soilgrids_service.py`?

Answer: ISRIC SoilGrids stores pH values in deci-units (integers) in its raster grid to save storage space. Dividing by 10 converts 68 to standard pH 6.8.

- Q2: Why use `predict_proba()` instead of `predict()`?

Answer: To output calibrated Top-3 suitability percentages (e.g. Rice 69.2%, Jute 18.4%, Maize 7.5%) rather than a single label.

- Q3: What is the role of `asyncio.gather()` in `recommend_from_map`?

Answer: It triggers Open-Meteo and ISRIC SoilGrids REST requests in parallel, cutting GIS telemetry latency in half (<300ms).